

COGNOMS:

[illegible]

NOM:

[illegible]

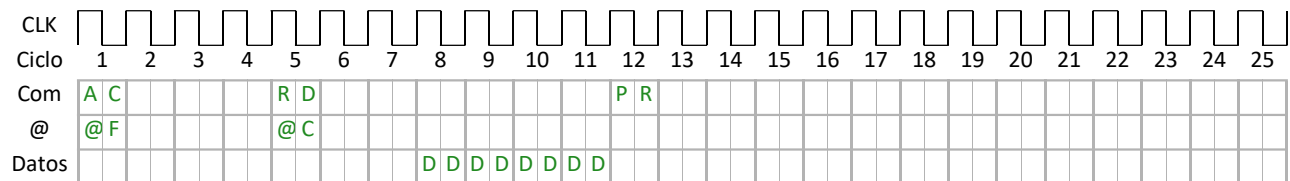
IMPORTANTE leer atentamente antes de empezar el examen: Escriba los apellidos y el nombre antes de empezar el examen. Escriba un solo carácter por recuadro, en mayúsculas y lo más claramente posible. Es importante que no haya tachones ni borrones y que cada carácter quede enmarcado dentro de su recuadro sin llegar a tocar los bordes. Use un único cuadro en blanco para separar los apellidos y nombres compuestos si es el caso. No escriba fuera de los recuadros.

Problema 1. (2.5 puntos)

Una **CPU** está conectada a una cache de instrucciones (**\$I**) y una cache de datos (**\$D**). El conjunto formado por **CPU+\$I+\$D** esta conectado a una memoria principal formada por un único módulo DIMM estándar de 16 GBytes. Este DIMM tiene 8 chips de memoria **DDR-SDRAM (Double Data Rate Synchronous DRAM)** de 1 byte de ancho cada uno. El DIMM esta configurado para leer/escribir ráfagas de 64 bytes (justo el tamaño de bloque de las caches). La latencia de fila es de 4 ciclos, la latencia de columna de 3 ciclos y la latencia de precarga de 1 ciclo.

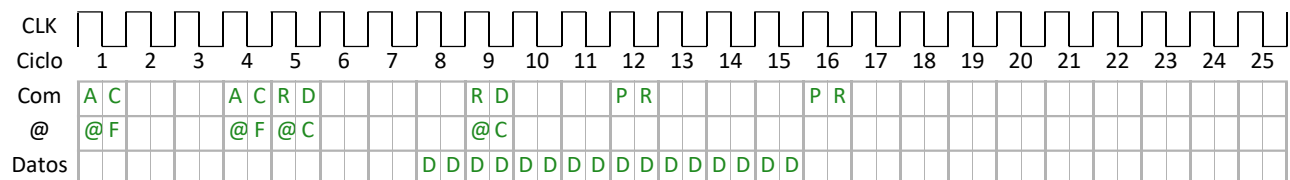
En los siguientes cronogramas, indica la ocupación de los distintos recursos de la memoria DDR: bus de datos, bus de direcciones y bus de comandos. En los cronogramas a-d supondremos que no hay ninguna página de DRAM abierta y al acabar la secuencia de accesos todas las paginas deben quedar cerradas.

a) **Rellena** el siguiente cronograma para una lectura de un bloque de 64 bytes de la DDR.

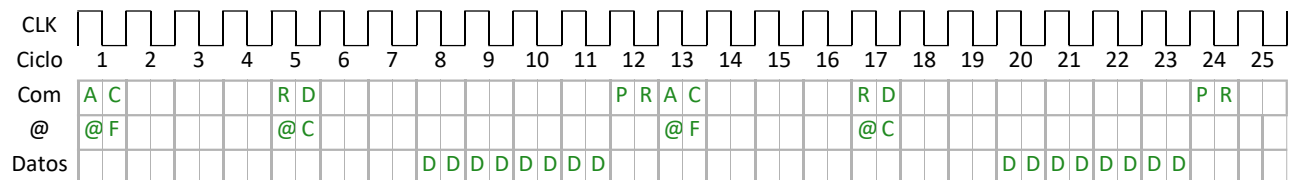


En ocasiones, es posible que el conjunto **CPU+\$I+\$D** solicite múltiples bloques a la DDR (por ejemplo porque se produzca un fallo simultáneamente en **\$I** y en **\$D**). Rellena los siguientes cronogramas para la lectura de dos bloques de 64 bytes en función de la ubicación de los dos bloques involucrados. El controlador de memoria envía los comandos necesarios a la DDR-SDRAM de forma que ambos bloques sean transferidos lo más rápidamente posible y se maximice el ancho de banda. El objetivo es minimizar el tiempo total.

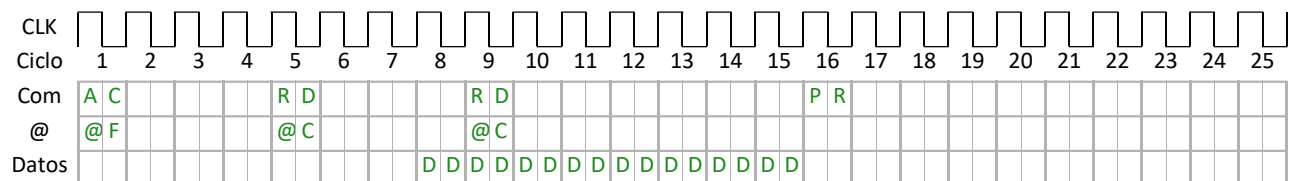
b) Ambos bloques están ubicados en bancos distintos.



c) Ambos bloques están ubicados en el mismo banco pero en páginas distintas.



d) Ambos bloques están ubicados en el mismo banco y en la misma página.



Queremos estudiar el comportamiento de la DRAM en un entorno real en función de las características de la memoria cache. Para ello hemos ejecutado el siguiente fragmento de código:

```
for (i=0; i<N; i=i+16)
    suma = suma + v[i]; // v[i] es un vector de doubles (8 bytes)
```

El código está almacenado en **\$I**, por lo que no habrá fallos en la cache de instrucciones, las variables *i*, *N* y *suma* están en registros y **\$D** está inicialmente vacía. Los elementos del vector *v* son de 8 bytes y los bloques de **\$D** son de 64 bytes.

Una versión compilada de este código ha generado 5 instrucciones para implementar el bucle. La CPU es un procesador superescalar fuera de orden que puede ejecutar una iteración por ciclo, generando por tanto una lectura a datos por ciclo.

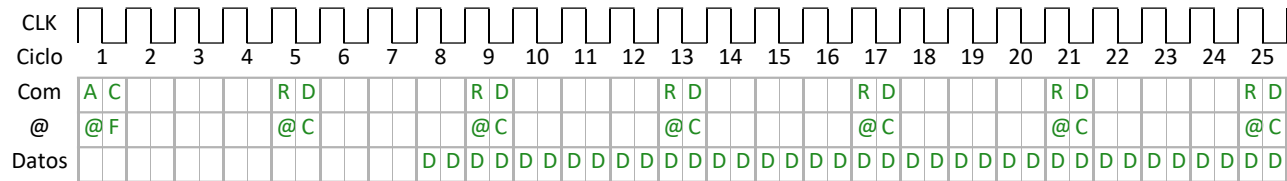
El controlador de memoria una vez ha abierto una página (fila) de DRAM no la cierra hasta que recibe un acceso que requiere una nueva página (distinta de la anterior). Sabemos que cada uno de los 8 chips del DIMM tiene páginas de 1KByte. Las direcciones de memoria están mapeadas de forma que: los 3 bits de menos peso de la dirección (2:0) seleccionan el chip y los 10 siguientes (12:3) seleccionan la columna.

e) **Calcula** la tasa de fallos de la cache de datos **\$D** y cuantos fallos puede servir la DRAM por cada página que abre.

Stride = 16*8 = 128 bytes. Tamaño bloque 64 bytes.
No hay localidad ni espacial ni temporal -> 100% de fallos -> **m=1**
1 pagina = 1KBytes -> 1 DIMM se abren 8Kbytes -> $8*1024/128 = 64$ fallos por pagina.

Sabemos que la cache de datos **\$D** es una cache no bloqueante con 16 **MSHRs**. En el siguiente cronograma dibuja los primeros 25 ciclos de los accesos a la DRAM generados por dicho bucle asumiendo que el primer acceso necesita abrir página.

f) **Primeros 25 ciclos de los accesos a la DRAM generados por el bucle.**



COGNOMS:

NOM:

Problema 2. (2.5 puntos)

Hemos simulado la ejecución de un programa en una CPU con un sistema de memoria en donde todos los accesos a memoria tardan 1 ciclo sean aciertos o fallos (denominaremos **CPU_{IDEAL}** a esta combinación simulada) y hemos obtenido que el programa se ejecuta en $12,5 \times 10^9$ ciclos, ejecuta 5×10^9 instrucciones, realiza 6×10^9 accesos a memoria y de estos, 500×10^6 son fallos de cache. La CPU funciona a 2,5GHz.

- a) **Calcula** el CPI, el numero de accesos por instrucción, la tasa de fallos, el tiempo de ejecución del programa en la **CPU_{IDEAL}** y el tiempo medio entre accesos en ciclos.

CPI =	$12,5 \times 10^9 \text{ ciclos} / 5 \times 10^9 \text{ instrucciones} = 2,5 \text{ c/i}$
Accesos por instrucción =	$6 \times 10^9 \text{ accesos} / 5 \times 10^9 \text{ instrucciones} = 1,2 \text{ a/i}$
Tasa de fallos =	$500 \times 10^6 \text{ fallos} / 6 \times 10^9 \text{ accesos} = 0,083 \text{ f/a} = 8,33\%$
Tiempo ejecución =	$12,5 \times 10^9 \text{ ciclos} / 2,5 \times 10^9 \text{ Hz} = 5 \text{ s}$
Tiempo medio entre accesos =	$12,5 \times 10^9 \text{ ciclos} / 6 \times 10^9 \text{ accesos} = 2,083 \text{ ciclos}$

Queremos integrar esta CPU con una cache unificada (instrucciones+datos) de mapeo directo. Esta cache no está segmentada y su tiempo de acceso es de 0,9 ns. Obsérvese que el tiempo de acceso es mayor que el tiempo de ciclo del procesador, por lo que al acceder a cache, el procesador se bloquea durante unos ciclos, y por tanto se produce una pequeña penalización respecto a la **CPU_{IDEAL}** (incluso en caso de acierto). En caso de que el acceso sea un fallo de cache, hay una penalización adicional de 20 ciclos más.

- b) **Calcula** los ciclos de penalización en caso de acierto.

Ciclos penalización acierto =	$\lceil 0,9 \times 10^{-9} \text{ s} * 2,5 \times 10^9 \text{ Hz} \rceil - 1 \text{ ciclo} = 2 \text{ ciclos}$
-------------------------------	--

Hemos cambiado la cache por otra nueva. En esta nueva los **ciclos de penalización por acierto son 1** y, por lo tanto, los ciclos de penalización por fallo (totales sobre la **CPU_{IDEAL}**) son 21.

- c) **Calcula** el tiempo de ejecución cuando ejecutamos el programa con la cache unificada.

$\text{ciclos} = 12,5 \times 10^9 \text{ ciclos} + 1 \text{ ciclos/acceso} * 6 \times 10^9 \text{ accesos} + 20 \text{ ciclos/fallo} * 500 \times 10^6 \text{ fallos} = 28,5 \times 10^9 \text{ ciclos}$
 $\text{Texe} = 28,5 \times 10^9 \text{ ciclos} / 2,5 \times 10^9 \text{ Hz} = 11,4 \text{ s}$

Nuestra CPU es capaz de continuar ejecutando instrucciones mientras se accede a la cache, sin embargo en el apartado c) bloqueamos la CPU en cada acceso para evitar lanzar un segundo acceso a la cache antes de que acabe el acceso anterior. Una posible mejora, que denominaremos control de bloqueos de cache, consiste en no bloquear la CPU en cada acceso, sino solamente si se inicia un acceso antes de que el anterior haya terminado. La CPU no soporta loads no bloqueantes, por lo que en caso de fallo siempre bloquearemos la CPU mientras se trae el bloque del siguiente nivel de la jerarquía (ciclos de penalización adicional). Sabemos que la probabilidad de realizar un acceso es la misma en todos los ciclos y es independiente de lo sucedido en ciclos anteriores. Durante los ciclos que no está bloqueada, la CPU se comporta exactamente igual que en el caso ideal por lo que el número medio de ciclos entre dos accesos será el mismo.

- d) **Calcula** la probabilidad de acceder a memoria en un ciclo determinado y la probabilidad de que al realizar un acceso la cache esté ocupada.

Probabilidad acceso en 1 ciclo =

$$1/\text{tiempo medio} = 1/2,083 = \mathbf{0,48}$$

Probabilidad de un acceso con cache ocupada =

como los fallos son bloqueantes, la cache sólo puede estar ocupada durante un ciclo (por el acceso anterior) -> **0,48**

En una cache organizada en bancos el acceso a cada banco es independiente, por lo que es posible acceder a un banco aunque otro este ocupado. Si organizamos nuestra cache en 4 bancos, una posible mejora, que denominaremos control de bloqueos de banco, consiste en bloquear la CPU solamente en caso de que accedamos a un banco ocupado. En nuestro caso, sabemos que en cada acceso la probabilidad de acceder a cualquiera de los 4 bancos es la misma, y que es independiente de los accesos anteriores. Como en el caso anterior, la CPU no soporta loads no bloqueantes, por lo que en caso de fallo siempre bloquearemos la CPU mientras se trae el bloque del siguiente nivel de la jerarquía (ciclos de penalización adicional).

- e) **Calcula** la probabilidad de que al realizar un acceso el banco accedido esté ocupado.

probabilidad de acceso con cache ocupada = 0,48

probabilidad de acceder mismo banco = $1/4$

probabilidad de acceder a banco ocupado = $1/4 * 0,48 = \mathbf{0,12}$

COGNOMS:

[illegible]

NOM:

[illegible]

Problema 3. (2.5 puntos)

Disponemos de un disco duro con 8 platos de 2 caras por cada plato y un radio útil para almacenar información de 3 cm. El brazo móvil permite una distancia entre pistas (incluyendo el grosor de las mismas) de $50\mu\text{m}$. Los sectores almacenan 357 bytes de información “bruta”, de los cuales 101 bytes corresponden a información de control. El disco gira a una velocidad de rotación (número de vueltas por minuto) de 5400 r.p.m. y se formatea con dos zonas de igual número de pistas pero con densidades distintas de sectores por pista (una zona tiene el doble de sectores por pista que la otra). El total de información “bruta” que puede almacenar una pista de la zona con más sectores por pista es de 188496 bits.

a) **Calcula** la capacidad “efectiva” del disco en bytes. **Escribe** el resultado en MB (potencias de 10).

```
n_pistas = 3x10^4μm/50μm = 600 pistas
tam_sector_bruta = 357 x 8 bits = 2856 bits
densidad_pista_zona1 = 188496 bits/2856 bits = 66 sectores por pista_zona1
densidad_pista_zona2 = 33 sectores por pista_zona2
tam_sector_data = 357 - 101 = 256 bytes

Capacidad = n_platosxn_carasx(n_pistas/2)x(np_zona1+np_zona2)x tam_sector_data
Capacidad = 8 x 2 x (600/2) x (66+33) x 256 B = 121651200 B = 121,65 MB
```

b) **Calcula** el tiempo de transferencia medio de un sector. Este tiempo sólo depende de la velocidad de rotación del disco y de la densidad de la pista. Asumir que el cabezal ya está situado al principio del sector. **Suponer** la misma probabilidad de que el sector sea de una pista de cualquiera de las dos zonas.

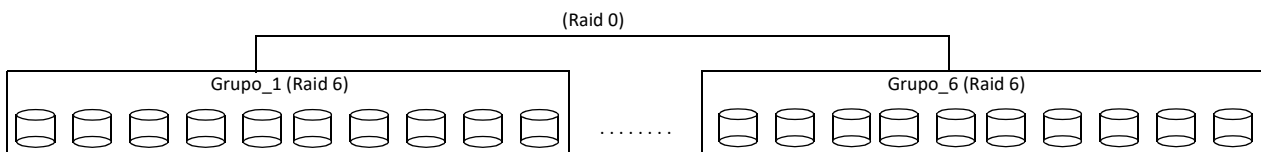
```

veloc_rotación = 5400 vueltas/60s = 90 vueltas por segundo
1/veloc_rotación = 11,1 ms para dar 1 vuelta (recorrer todos los sectores de una
pista)
11,1 ms / 66 = 0,168 ms para un sector la zonal
11,1 ms / 33 = 0,336 ms para un sector la zona2

t_promedio = (0,5 x 0,168 ms) + (0,5 x 0,336 ms) = 0.25 ms
(tb. t promedio = 1,5 x 0,168 ms = 0.25 ms)

```

Disponemos de 60 discos físicos con capacidad de 120 GBytes de capacidad por disco, y queremos montar con ellos un disco lógico con la configuración RAID6+0 con 6 grupos de 10 discos en cada grupo:



c) **Calcula** la cantidad de información útil que puede almacenar este sistema RAID6+0. Escribe el resultado en TB (potencias de 10). **Calcula** el porcentaje de información redundante que hay en este sistema RAID6+0.

$G=6$, $N=60$ y 10 discos en cada G

Información útil = $(10 - 2) \times G = 48$ discos $\times$ 120 GBytes = **5,76 TBytes**

Porcentaje de información redundante = $(2 \times G) / N = 12 / 60 =$ **20%**

Hemos medido la ejecución de un programa en nuestro PC de sobremesa con un solo procesador y un solo disco y vemos que su tiempo de ejecución es de 200 horas. Este programa consta de dos fases, una de E/S que supone el 80% del tiempo de ejecución y otra de cálculo que no podemos paralelizar. Los datos de la intensiva E/S que maneja el programa son muy críticos para nosotros y, aún sabiendo que el sistema RAID6+0 es complejo y nos va a resultar un poco caro, hemos decidido modificar nuestro PC usando su único procesador y el sistema RAID6+0 de 60 discos del apartado anterior. Sabemos que en nuestro programa las 3/4 partes del tiempo de la fase de E/S se gasta en lecturas y el resto en escrituras, que todos son accesos aleatorios, y que hay suficientes accesos como para saturar el ancho de banda de los discos.

- d) **Calcula** la ganancia en tiempo con respecto al PC de sobremesa (un procesador y un disco) que obtendremos si el programa lo ejecutamos en el nuevo entorno con un procesador y un RAID6+0 formado por los 60 discos.

```
T_calculo = 0.20 x 200h = 40h
T_e/s = 200h - 40h = 160h de las cuales:

3/4: 120h son para lecturas y 1/4: 40h son para escrituras

T_nuevo = 40h + 120h/60 + 40h/(60/6) = 40 + 2 + 4 = 46 horas

Ganancia = 200 / 46 = 4,34
```

[illegible][illegible]

Problema 4. (2.5 puntos).

Los procesadores CISC de la familia x86 traducen internamente las instrucciones de lenguaje máquina x86 a micro operaciones (que llamaremos μ ops). En una posible implementación de x86 tenemos las siguientes μ ops:

Tipo	Descripción	Ejemplo
LOAD	Rd <-- M[Rb+Ri*s+despl]	LOAD %R1 <-- M[%EAX+%R2*4-37]
STORE	M[Rb+Ri*s+despl] <-- Rf	STORE M[%EAX+100] <-- %EBX
MOV	Rd <-- Rf Rd <-- inm	MOV %R3 <-- %ESI MOV %EAX <-- 5
ARITMÉTICO - LÓGICAS	Rd <-- Rf1(op)Rf2 Rd <-- Rf1(op)inm	ADDL %R4 <-- %EAX + %EBX MULL %ECX <-- %R4 * 38
COMPARACIÓN	Flags <-- Rf1-Rf2 Flags <-- Rf1-inm	CMPL %EAX, %EBX CMPL %R1,12
SALTO	Los mismos de x86	JNE loop

Sólo las μ ops LOAD, STORE pueden acceder a memoria (usando los mismos modos de direccionamiento de x86), el resto de μ ops sólo pueden usar registros o inmediatos como operandos. Además de los registros visibles de x86 disponemos de registros adicionales para almacenar valores intermedios (%R0, %R1, ..., %R7). Recordad que el registro %EIP (contador de programa) almacena la dirección de la siguiente instrucción a ejecutar.

a) **Traducid** la siguiente secuencia de instrucciones en x86 a la correspondiente secuencia de μ ops.

```
PUSHL 100(%EBX)
CALL subr
ADDL $4, %ESP
ADDL %EAX, (%EDI)
```

```
LOAD %R1 <-- M[%EBX+100]
%ESP <-- %ESP - 4
STORE M[%ESP] <-- %R1
%ESP <-- %ESP - 4
M[%ESP] <-- %EIP
%EIP <-- (@subr)
%ESP <-- %ESP + 4
LOAD %R2 <-- M[%EDI]
ADDL %R2 <-- %R2 + %EAX
STORE M[%EDI] <-- %R2
```

A continuación, se muestra una secuencia de instrucciones en x86 y al lado su correspondiente traducción a μops (suponed que $\%ECX$ vale inicialmente 10^9 , el bucle da 10^9 iteraciones).

<pre> LOOP: MOVL (%EBX,%ECX,4),%EAX ADDL %EAX, (%EDI,%ECX,4) DECL %ECX JNE LOOP </pre>	<pre> LOOP: LOAD %EAX <-- M[%EBX+ECX*4] LOAD %R1 <-- M[%EDI+ECX*4] ADDL %R1 <-- %EAX + %R1 STORE M[%EDI+ECX*4] <-- %R1 SUBL %ECX <-- %ECX - \$1 JNE LOOP </pre>
---	--

- b) Suponiendo que la frecuencia del procesador es 2 GHz y que el programa se ejecuta en 1,2 segundos, **calculad** el CPI y el $CP_{\mu op}$ (ciclos por μop).

Se ejecutan: $4 \cdot 10^9$ instrucciones de x86 y $6 \cdot 10^9 \mu ops$

$T_{exe} = N \cdot CPI \cdot T_c \rightarrow CPI = T_{exe} \cdot Freq / N$

$CPI = 1,2 \cdot 2 / 4 = 0,6 \text{ c/i}$

$CP_{\mu op} = 1,2 \cdot 2 / 6 = 0,4 \text{ c}/\mu op$

Vamos a utilizar una versión superescalar del mismo procesador para una aplicación P de cálculo intensivo. Con un solo procesador la aplicación tarda en ejecutarse 100 horas. Esta aplicación tiene un 80% totalmente paralelizable (aquí se alcanza una velocidad de 2,5 GFLOPs) y un 20% que ha de ejecutarse secuencialmente. El procesador (CPU) consume 100W.

Disponemos de 2 posibles sistemas paralelos:

- Un multiprocesador con 10 CPUs
- Un computador con 1 CPU y 4 GPUs. La CPU se ha de utilizar para la parte secuencial, y las GPUs para la parte paralela. Las GPUs alcanzan con este código los 50 GFLOPs cada una. Cada GPU consume 300 W.

- c) **Calculad** el tiempo de ejecución de la aplicación P en los 2 sistemas paralelos.

Multiprocesador:

- Parte secuencial: 20h

- Parte paralela: $80h / 10 \text{ CPUs} = 8h \rightarrow \text{Total } 28 \text{ horas}$

Sistema con 4 GPUs

- Parte secuencial: 20h

- Parte paralela: Las GPUs son 20 veces más rápidas

$80h / 20 / 4 \text{ GPUs} = 1h \rightarrow \text{Total } 21 \text{ horas}$

- d) **Calculad** la energía consumida (CPUs y GPUs) por los 2 sistemas paralelos y el sistema con 1 sólo procesador (podéis suponer que cuando no se está utilizando una CPU o una GPU el consumo es nulo).

1) Secuencial: $E = 100 \cdot 3600 \cdot 100 = 36 \times 10^6 \text{ J}$

2) Multiprocesador 10 CPUs: $E = 20 \cdot 3600 \cdot 100 + 8 \cdot 3600 \cdot 100 \cdot 10 = 36 \times 10^6 \text{ J}$

3) 1CPU + 4 GPUs: $E = 20 \cdot 3600 \cdot 100 + 1 \cdot 3600 \cdot 300 \cdot 4 = 11,52 \times 10^6 \text{ J}$